

Solutions - Debugging Techniques

Worked solutions and model answers for all 8 exercises, including measured example results and the completed code for the steps that asked you to fill something in.

STEP 1 OF 8 - SOLUTION

Reproduce and Define the Symptom

The code below is unchanged from the exercise - Step 1 is about observing and describing the symptom, not modifying the code.

- Typical result: repeated runs cluster around 0.03-0.05 seconds on a typical laptop, with occasional higher outliers caused by background activity.
- Model problem statement: "process() takes about 40ms to process 300,000 integers, which is too slow for a code path with a 10ms budget."
- Model environment note: quad-core laptop, otherwise idle, Python 3.12, single run of 300,000 items - written down so later comparisons are apples-to-apples.

```
solution_step1.py

import time

def process(values):
    total = 0
    for value in values:
        total += (value * value) % 97
    return total

values = list(range(300_000))

start = time.perf_counter()
result = process(values)
elapsed = time.perf_counter() - start

print("result:", result)
print("elapsed:", elapsed)
```

STEP 2 OF 8 - SOLUTION

Build a Controlled Diagnostic Benchmark

The benchmark code is unchanged - the exercise asked for reasoning about the method, not a code change.

- Why a changed workload invalidates comparison: a timing difference could then come from the different input, not from any real improvement - the two numbers would no longer be comparable.
- Noise sources: other running applications, CPU frequency/thermal throttling, virtual machine or container scheduling, and disk or swap activity.
- `min(times)` approximates the best-case cost with system noise stripped out; `avg(times)` is more useful once consistency, not just best case, also matters.

```
solution_step2.py

import time
from step1_reproduce import process

def benchmark(fn, runs=10):
    times = []
    result = None
    for _ in range(runs):
        start = time.perf_counter()
        result = fn()
        times.append(time.perf_counter() - start)
    return result, times

values = list(range(300_000))
result, times = benchmark(lambda: process(values))

print("min:", min(times))
print("avg:", sum(times) / len(times))
print("result:", result)
```

STEP 3 OF 8 - SOLUTION

Inspect Suspicious State with pdb

The code is unchanged. Comments below show the actual pdb session, confirmed by running it.

- At the breakpoint: p index -> 3, p value -> -4, p total -> 14 (the sum of squares of 1, 2, and 3 so far).
- where shows process() was called directly from module level, at the print(process(values)) line.
- Conditional debugging is preferable because it lets the loop run at full speed until the one case actually worth investigating, instead of pausing on every iteration.

```
solution_step3.py

def process(values):
    total = 0
    for index, value in enumerate(values):
        if value < 0:
            breakpoint()
        total += value * value
    return total

values = [1, 2, 3, -4, 5]

print(process(values))

# (Pdb) p index      -> 3
# (Pdb) p value      -> -4
# (Pdb) p total      -> 14
# (Pdb) c            -> 55
```

STEP 4 OF 8 - SOLUTION

Add Targeted Timing and Call-Count Diagnostics

The code is unchanged - the task was to interpret the numbers it produces.

- Typical result: calls: 50000, measured transform time: roughly 0.01-0.02 seconds total - under 1 microsecond per call on average.
- Since each call is already this cheap, call frequency (not per-call cost) is what would dominate if this function were invoked far more often.
- `perf_counter()` itself has a small overhead; when timing something this fast, that overhead is a meaningful fraction of what gets measured.
- If the operation became much faster still, aggregate timing across a batch (as done here) rather than timing every single call, to keep the overhead ratio low.

```
solution_step4.py

import time

calls = 0

def expensive_transform(value):
    global calls
    calls += 1
    start = time.perf_counter()
    result = str(value).strip().lower()[::-1]
    elapsed = time.perf_counter() - start
    return result, elapsed

values = range(50_000)
total_time = 0.0

for value in values:
    _, elapsed = expensive_transform(value)
    total_time += elapsed

print("calls:", calls)
print("measured transform time:", total_time)
```

STEP 5 OF 8 - SOLUTION

Diagnose Repeated Work in a Hot Path

The code is unchanged - the task was to reason about the cost, not to fix it yet.

- Cost estimate: 100,000 values, each scanning up to 10,000 target items - on the order of hundreds of millions of comparisons in the worst case.
- Hypothesis: converting targets to a set would make each membership check $O(1)$ on average instead of $O(n)$, removing that scaling problem entirely.
- This hypothesis is confirmed by measurement in Step 7, not assumed - the implementation is deliberately left unchanged here.

```
solution_step5.py

def count_matches(values, targets):
    checks = 0
    count = 0
    for value in values:
        checks += 1
        if value in targets:
            count += 1
    return count, checks

values = list(range(100_000))
targets = list(range(0, 100_000, 10))

result, checks = count_matches(values, targets)

print("matches:", result)
print("membership checks:", checks)
```

STEP 6 OF 8 - SOLUTION

Investigate Memory Growth

The code is unchanged - the task was to interpret the tracemalloc output.

- Allocation site: the line inside the loop that builds each dict; the payload string (5x the length of the id) is the larger contributor.
- The objects remain alive because cache is a module-level list that is never cleared - every dict stays reachable for the life of the program.
- This is legitimate retention, not a leak: len(cache) matches the loop count exactly (50,000), with nothing meant to be discarded along the way.
- Bounded alternative: cap the list with a collections.deque(maxlen=...), or persist entries to disk incrementally instead of holding everything in memory at once.

```
solution_step6.py

import tracemalloc

cache = []

tracemalloc.start()
snapshot_before = tracemalloc.take_snapshot()

for i in range(50_000):
    cache.append({"id": i, "payload": str(i) * 5})

snapshot_after = tracemalloc.take_snapshot()

for stat in snapshot_after.compare_to(
    snapshot_before, "lineno",
)[:10]:
    print(stat)

print("retained objects:", len(cache))

tracemalloc.stop()
```

STEP 7 OF 8 - SOLUTION

Fix the Root Cause and Validate

The code is unchanged - it already is the fix. Example measured output is shown as comments.

- Root cause: `count_matches_list` is $O(n \times m)$ - for each of 200,000 values, up to 20,000 targets are scanned.
- `count_matches_set` makes each check $O(1)$ on average via hashing, so total cost drops to roughly $O(n)$.
- Semantics preserved: `set(targets)` contains exactly the same elements as the list, so membership results are identical - `assert a == b` proves this for this run.
- Example measured result on this run: baseline ~ 24.7 s, optimised ~ 0.006 s, speedup ~ 4150 x. Exact numbers vary by machine.

```
solution_step7.py

import time

def count_matches_list(values, targets):
    return sum(1 for v in values if v in targets)

def count_matches_set(values, targets):
    target_set = set(targets)
    return sum(1 for v in values if v in target_set)

values = list(range(200_000))
targets = list(range(0, 200_000, 10))

start = time.perf_counter()
a = count_matches_list(values, targets)
baseline = time.perf_counter() - start

start = time.perf_counter()
b = count_matches_set(values, targets)
optimised = time.perf_counter() - start

assert a == b

print("baseline:", baseline)      # e.g. 24.7
print("optimised:", optimised)    # e.g. 0.006
print("speedup:", baseline / optimised)  # e.g. 4150.0
```

STEP 8 OF 8 - SOLUTION

Produce a Debugging and Optimisation Report

The report below is completed with example values measured in Step 7 - this is what a finished report looks like.

- $\text{speedup} = \text{baseline_seconds} / \text{optimised_seconds}$, calculated directly from the Step 7 measurements.
- Test environment: state the machine, OS, and Python version, plus the exact workload size (200,000 values, 20,000 targets) - anyone should be able to repeat this run.
- Evidence for the root cause: Step 5's checks counter confirmed one membership check per value; Step 7's timing directly measured the $O(n)$ vs $O(1)$ difference that Step 5 predicted.
- Limitations: a single-machine, single-run measurement; a follow-up could test with a very small target set, where the set-conversion overhead itself might start to dominate.

```
solution_step8.py

report = {
    "symptom": "membership-heavy workload is slow",
    "baseline_seconds": 24.7,
    "diagnostic_evidence": [
        "high repeated membership cost",
        "linear list membership",
        "large number of checks",
    ],
    "root_cause": "repeated  $O(n)$  membership searches",
    "change": "use a set for repeated membership tests",
    "optimised_seconds": 0.006,
    "speedup": 4150.0,
    "correctness_verified": True,
}

print(report)
```